

EXPERIMENT 7

AIM: Program development using WHILE LOOPS, numeric FOR LOOPS, nested loops using ERROR Handling, BUILT-IN Exceptions, USE defined Exceptions, RAISE- APPLICATION ERROR

SOLUTION:

1. WHILE LOOP

A **WHILE loop** repeats a block of statements **as long as a condition is TRUE**.

Syntax

WHILE condition LOOP

statements;

END LOOP;

2. Numeric FOR LOOP

A **Numeric FOR loop** runs for a **fixed number of times** automatically.

Syntax

FOR i IN start..end LOOP

statements;

END LOOP;

3. NESTED LOOP

A **loop inside another loop** is called a nested loop.

Example:

FOR i IN 1..3 LOOP

FOR j IN 1..2 LOOP

statements;

END LOOP;

END LOOP;

4. BUILT-IN EXCEPTIONS

Some common built-in exceptions:

Exception	Meaning
NO_DATA_FOUND	No row found
TOO_MANY_ROWS	More than one row returned
ZERO_DIVIDE	Division by zero
VALUE_ERROR	Conversion error

5. USER-DEFINED EXCEPTION

We create our own exception using:

```
my_exception EXCEPTION;
```

Then raise it using:

```
RAISE my_exception;
```

6. RAISE_APPLICATION_ERROR

Used to display **custom error messages**.

```
RAISE_APPLICATION_ERROR(-20001, 'Your error message');
```

(Error number must be between -20000 to -20999)

Program:

```
SET SERVEROUTPUT ON;
```

```
DECLARE
```

```
    i NUMBER := 1;
```

```
    j NUMBER;
```

```
    v_num NUMBER := 1;
```

```
    user_error EXCEPTION; -- user-defined exception
```

```
BEGIN
```

```
    -- WHILE LOOP
```

```
    WHILE i <= 3 LOOP
```

```
        DBMS_OUTPUT.PUT_LINE('WHILE LOOP COUNT: ' || i);
```

```
        i := i + 1;
```

```
    END LOOP;
```

```
    DBMS_OUTPUT.PUT_LINE('-----');
```

```
    -- NUMERIC FOR LOOP + NESTED LOOP
```

```
    FOR x IN 1..3 LOOP
```

```
        DBMS_OUTPUT.PUT_LINE('TABLE OF ' || x);
```

```
        FOR y IN 1..5 LOOP
```

```
            DBMS_OUTPUT.PUT_LINE(x || ' * ' || y || ' = ' || (x*y));
```

```
        END LOOP;
```

```
        DBMS_OUTPUT.PUT_LINE('-----');
```

```
    END LOOP;
```

```
    -- BUILT-IN EXCEPTION TEST
```

```
    IF v_num = 0 THEN
```

```

        RAISE ZERO_DIVIDE;

    END IF;

    -- USER-DEFINED EXCEPTION

    IF v_num < 0 THEN

        RAISE user_error;

    END IF;

    DBMS_OUTPUT.PUT_LINE('PROGRAM COMPLETED SUCCESSFULLY');

EXCEPTION

    WHEN ZERO_DIVIDE THEN

        DBMS_OUTPUT.PUT_LINE('BUILT-IN EXCEPTION: Division by zero error');

    WHEN user_error THEN

        RAISE_APPLICATION_ERROR(-20001, 'User defined error: Number is negative');

    WHEN OTHERS THEN

        DBMS_OUTPUT.PUT_LINE('UNKNOWN ERROR OCCURRED');

END;

/

```

Output:

```

WHILE LOOP COUNT: 1
WHILE LOOP COUNT: 2
WHILE LOOP COUNT: 3

```

TABLE OF 1

```

1 * 1 = 1
1 * 2 = 2
1 * 3 = 3
1 * 4 = 4
1 * 5 = 5

```

TABLE OF 2

```

2 * 1 = 2
2 * 2 = 4
2 * 3 = 6
2 * 4 = 8

```

$$2 * 5 = 10$$

TABLE OF 3

$$3 * 1 = 3$$

$$3 * 2 = 6$$

$$3 * 3 = 9$$

$$3 * 4 = 12$$

$$3 * 5 = 15$$

PROGRAM COMPLETED SUCCESSFULLY

PL/SQL procedure successfully completed.